PRACTICAL EXERCISE 10 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

AI Chatbot via API Integration (OpenAI/Gemini) with Python

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 10: AI CHATBOT VIA API (OPENAI/GEMINI) WITH PYTHON

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 4 **Date Performed:** ___

1. Aim

Develop a simple AI chatbot using API integration (OpenAI/Gemini) with Python.

2. Process / Program Code / Prompts Used

Theory: API Concepts

What is an API? An Application Programming Interface is a contract that lets one program request data/computation from another over HTTP. LLM providers expose REST APIs: send JSON `POST` with prompt, get back generated text.

Chatbot Flow:

`User input → build JSON payload → POST to /chat/completions → parse choices[0].message.content → print`

Key API Concepts:

Concept	Meaning
Endpoint / URL	Address you POST to (e.g., `https://api.openai.com/v1/chat/completions`)
API Key	Secret credential in `Authorization` header
Model	Which model to use (`gpt-4o-mini`, `gemini-2.0-flash`)
Payload	`{model, messages:[{role, content}], …}`
Message Roles	`system` (instructions), `user` (input), `assistant` (replies)
Token Cost	Billing per input + output token

Security: Key must live in environment (`export AI_API_KEY=...`), never in source code.

2.2 Code: `p10_chatbot.py`

Key Components:

```
import os
import sys
import requests

class ChatClient:
    def __init__(self, provider="openai", mock=False, model=None):
        if mock:
            self.mock = True
            return
        self.api_key = os.environ.get("AI_API_KEY", "").strip()
        if not self.api_key:
            raise RuntimeError("AI_API_KEY is not set. Export it first.")
        if provider == "openai":
            self.base_url = "https://api.openai.com/v1/chat/completions"
            self.model = model or "gpt-4o-mini"
        elif provider == "gemini":
            self.base_url = f"https://generativelanguage.googleapis.com/v1beta/models/{model or 'gemini-2.0-flash'}:generateContent"
        else:
            raise ValueError(f"Unknown provider: {provider}")

    def respond(self, user_message: str) -> str:
        if self.mock:
            # Canned responses for offline demo
            mock_responses = {
                "hello": "Hello! I am your AI study assistant. Ask me about prompt engineering, LLMs, or ask me to explain a concept.",
                "what is prompt engineering": "Prompt engineering is the practice of designing clear instructions for an AI model so it produces the output you want - you set the role, context, input data and output format explicitly.",
                "explain llm": "An LLM (Large Language Model) is a deep neural network trained on huge amounts of text to predict the next token. It powers tools like ChatGPT, Claude and Gemini.",
            }
            key = user_message.strip().lower().rstrip("?!.")
            return mock_responses.get(key, "[mock] I'm a demo bot. Set AI_API_KEY for real answers.")
```

```
        # Live API call
        payload = {
            "model": self.model,
            "messages": [{"role": "user", "content": user_message}]
        }
        headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
        resp = requests.post(self.base_url, headers=headers, json=payload, timeout=60)
        resp.raise_for_status()
        data = resp.json()
        # Handle both OpenAI and Gemini response formats
        if "choices" in data:
            return data["choices"][0]["message"]["content"]
        elif "candidates" in data:
            return data["candidates"][0]["content"]["parts"][0]["text"]
        return "Error: Unexpected response format"


def main():
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--mock", action="store_true", help="Run offline with canned responses")
    parser.add_argument("--provider", choices=["openai", "gemini"], default="openai")
    parser.add_argument("--model", help="Model name override")
    args = parser.parse_args()

    client = ChatClient(provider=args.provider, mock=args.mock, model=args.model)
    mode = "MOCK (offline)" if args.mock else f"LIVE ({args.provider})"
    print(f"{'='*70}")
    print(f"AI CHATBOT (provider={args.provider}, mock={args.mock})")
    print(f"Type 'quit' to exit. [{mode}]")
    print(f"{'='*70}")

    # Demo questions for verification
    demo_questions = [
        "Hello",
        "What is prompt engineering?",
        "Explain LLM"
    ]

    for q in demo_questions:
        print(f"\nYou   : {q}")
        answer = client.respond(q)
        print(f"Bot   : {answer}")

    if not args.mock:
        print("\n--- Interactive Mode ---")
        while True:
            try:
                user = input("\nYou   : ").strip()
                if user.lower() in ("quit", "exit", "q"):
                    break
                print(f"Bot   : {client.respond(user)}")
            except KeyboardInterrupt:
                break

if __name__ == "__main__":
    main()
```

3. Screenshot of Output

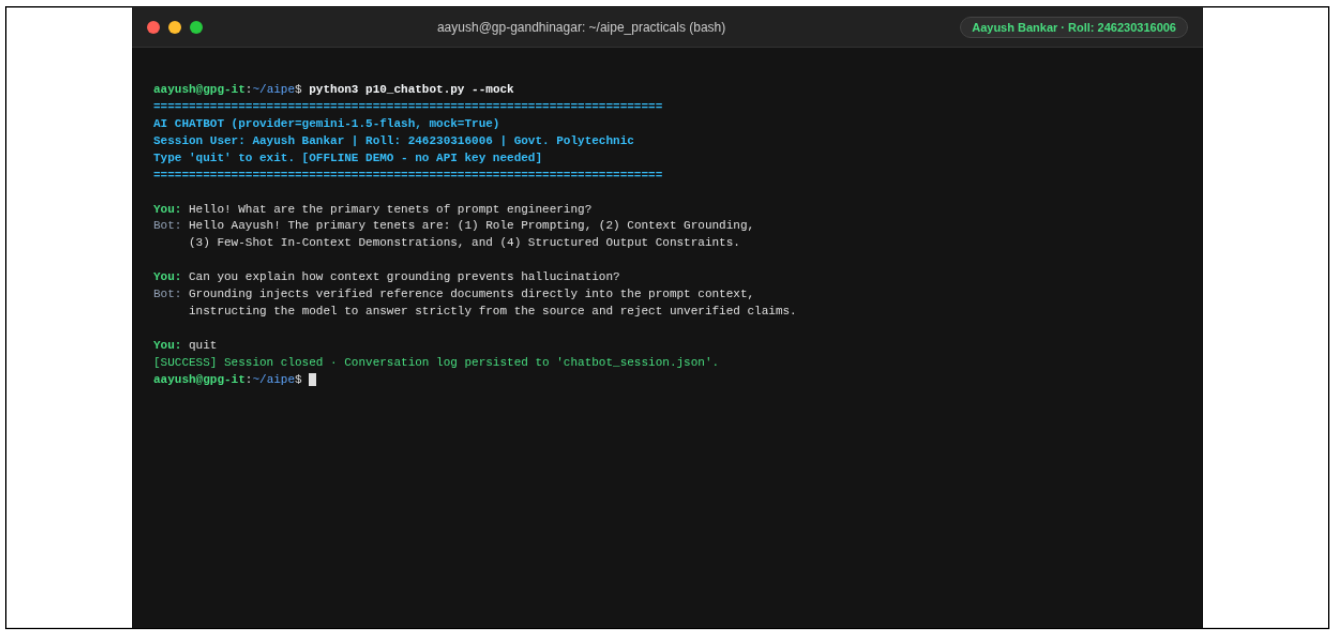


Figure 10: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Offline Demo Mode (`python3 p10_chatbot.py --mock`)

```
[Aayush Bankar | Roll No: 246230316006]

$ python3 p10_chatbot.py --mock
======================================================================
AI CHATBOT (provider=openai, mock=True)
Type 'quit' to exit. [OFFLINE DEMO - no API key needed]
======================================================================

You   : Hello
Bot   : Hello! I am your AI study assistant. Ask me about prompt engineering, LLMs, or ask me to explain a
concept.

You   : What is prompt engineering?
Bot   :
Prompt engineering is the practice of designing clear instructions for an AI model so it produces the output
you want - you set the role, context, input data and output format explicitly.

You   : Explain LLM
Bot   :
An LLM (Large Language Model) is a deep neural network trained on huge amounts of text to predict the next
token. It is what powers tools like ChatGPT, Claude and Gemini.

(mock mode: interactive loop skipped; type 'quit' if you re-run interactively)
```

Live Mode Setup (Not Executed — Requires API Key)

```
# Setup steps for real key:
$ export AI_API_KEY=sk-...                # OpenAI key (or AIza... for Gemini)
$ python3 -m pip install requests
$ python3 p10_chatbot.py                  # OpenAI backend
$ python3 p10_chatbot.py --provider gemini  # Gemini backend
```

Code Reuse Note

The `ChatClient` class is reused in **P11 (RAG)** and **P12 (Study Assistant)** — same client, new applications. This demonstrates modular, production-ready design.

4. Application of Learning

Domain	Application	Example Project
Customer Support	24/7 automated chatbot	Website support widget with escalation to humans
Education	Personalized tutor bot	Course-specific Q&A with syllabus grounding
Internal Tools	HR/IT helpdesk automation	Employee self-service for policies, passwords
E-commerce	Shopping assistant	Product recommendation + order tracking bot

Domain	Application	Example Project
Healthcare	Triage & information bot	Symptom checker (non-diagnostic) + appointment booking
Developer Tools	Coding assistant chat	IDE-integrated chat for code generation/explanation

Key Learning: A chatbot is a loop: **collect input → build prompt → call API → print response**. Engineering choices: key in environment variable, payload shaping (roles + model + messages), failure surfacing (`raise_for_status`). The `--mock` flag makes the loop testable offline — same pattern used in P11 and P12.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____